

CSC 212: Data Structures and Abstractions

05: Dynamic Arrays (part 1)

Prof. Marco Alvarez

Department of Computer Science and Statistics
University of Rhode Island

Fall 2026



Where we are?

- Last lecture: **asymptotic notation**
 - ✓ O, Ω, Θ : compare growth rates, not stopwatch times
 - ✓ drop constant factors, drop lower-order terms
- This lecture: use it to **design** a data structure
 - ✓ why arrays are fast, what they cost to change, and why the obvious way to grow one is a trap

2

Unknown lengths

- A chatbot (LLM) streams its answer one token at a time
 - ✓ a token is a word or a piece of a word
 - ✓ nobody knows how long the reply is until the model stops
- Our job: store the tokens as they arrive

```
#include <iostream>
#include <string>

int main() {
    std::string tokens[1000]; // how big should this be?
    size_t n = 0;
    std::string t;
    while (std::cin >> t) // one token at a time
        tokens[n++] = t; // what happens when n reaches 1000?
    std::cout << n << " tokens\n";
}
```

3

Q5-1

- The reply turns out to be 1,500 tokens long. What happens?
- The program does not compile
 - The array grows automatically to fit
 - Writing past `tokens[999]` is *undefined behavior*: the program may crash, or silently corrupt memory
 - C++ throws a `std::out_of_range` exception
 - None of the above

4

What is needed?

- An array whose size is decided **at run time**, and can **change** (grow or shrink)
 - ✓ requires efficient operations

operation	desired cost
read or write $A[i]$	$\Theta(1)$
append at the end	$\Theta(1)$ per append, on average over many appends
memory used	$\Theta(n)$ for n elements

- ✓ need to understand how memory works and how to grow the array efficiently

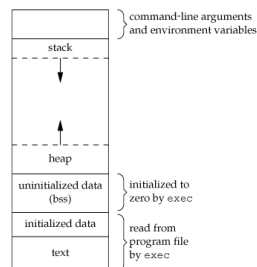
5

Memory and pointers

Memory layout

- A **process** is an **instance** of a running program with its own memory, laid out in regions

- ✓ **text (code)**: the compiled instructions, read-only
- ✓ **data**: global and static variables, size fixed at compile time
- ✓ **heap**: memory requested at run time with `new`, grows upward
- ✓ **stack**: local variables and function call frames, grows downward



7

Stack vs heap

	Stack	Heap
size decided	at compile time	at run time
freed	automatically, when the function returns	only when the program says so (<code>delete[]</code>)
typical use	small locals, fixed-size arrays	data whose size or lifetime is not known in advance

C++ (C++20 + GNU extensions)
[known limitations](#)

```
1 void foo(int n) {  
2   int a[10];           // stack: size fixed a  
3   int* p = new int[n];  // heap: size chosen  
4   a[0] = p[0] = 1;  
5   delete[] p;          // heap: freed only w  
6 }                       // stack: a is freed  
7  
8 int main() {  
9   foo(4);  
10 }
```

[Edit this code](#)

→ line that just executed
→ next line to execute

<< First < Prev Next > Last >>

Step 5 of 8

Stack Heap

main

foo(int)

int
4

array

a [int int int int int int int int int int]

1 2 3 4 5 6 7 8 9 10

pointer to int

p

Note: ? refers to an uninitialized value
C/C++ details: none (default view)

<https://pythontutor.com/>

8

Q5-2

▸ Which statement is true?

- A. `array` and the block `ptr` points to are both on the stack
- B. `array` is on the stack; the block `ptr` points to is on the heap, and `main` must free it
- C. `sort` receives `arr`, so `sort` must call `delete[] arr`
- D. `sort(array, 100)` does not compile, because `array` is not a pointer
- E. None of the above

```
void sort(int *arr, int size) {  
    // sorting logic here  
}  
  
int main() {  
    int array[100];  
    int *ptr;  
    ptr = new int[100];  
    sort(ptr, 100);  
    sort(array, 100);  
    delete[] ptr;  
    return 0;  
}
```

9

Ownership

▸ Every heap block has an **owner**: the code responsible for freeing it

▸ every `new[]` gets exactly one matching `delete[]`

▸ What can go wrong?

```
int* p = new int[4];  
int* q = p;           // two pointers, one block  
delete[] p;           // block freed  
q[0] = 7;             // dangling: undefined behavior
```

undefined behavior means anything can happen: a crash, a wrong answer, or a program that does not work

mistake	name	result
never call <code>delete[]</code>	memory leak	memory is lost until the program ends
call <code>delete[]</code> twice	double free	undefined behavior
use memory after <code>delete[]</code>	dangling pointer	undefined behavior

10

Q5-3

▸ What is wrong with this program?

```
int* make() {  
    int* p = new int[10];  
    return p;  
}  
  
int main() {  
    int* a = make();  
    int* b = a;  
    delete [] a;  
    delete [] b;  
}
```

- A. A memory leak: `make` never frees `p`
- B. A double free: `a` and `b` point to the same block, which is freed twice
- C. A dangling pointer: `p` stops existing when `make` returns, so `a` is invalid
- D. Nothing: every `new[]` has a `delete[]`
- E. None of the above

11

Arrays

Contiguous memory

- An **array** is a **contiguous** block: element i sits right after element $i - 1$

0	1	2	3				n-1
A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]
1000	1004	1008	1012	1016	1020	1024	1028

- The address of $A[i]$ is computed, never searched for

✓ address of $A[i] = \text{base} + i \times \text{sizeof}(T)$

```
int A[8]; // base address 1000, sizeof(int) = 4
```

✓ calculate the address of $A[4]$

13

Q5-4

- `int A[10]` starts at address 2000, and `sizeof(int)` is 4. What is the address of $A[7]$?

- A. 2007
- B. 2070
- C. 2032
- D. 2028
- E. None of the above

14

Contiguous memory

- $\Theta(1)$ element access and cache locality
 - ✓ hardware adds a second reward: memory moves in **blocks** (cache lines, typically 64 bytes = 16 ints)
 - ✓ reading $A[i]$ brings its neighbors along for free
- Sum an array of 10^7 ints
 - in order $\Rightarrow 0.009$ s
 - in random order $\Rightarrow 0.076$ s
 - measured on an *Intel Core i7-8700B*, clang -O2, median of three runs
 - ✓ same $\Theta(n)$ work, about **8x slower**, only because of the order

Principle: contiguous memory makes indexing constant time, and scanning fast

15

Array operations

- Consider an array of n elements, with room to spare

operation	elements moved	cost
read or write $A[i]$	0	$\Theta(1)$
append at the end (<code>push_back</code>)	0	$\Theta(1)$
remove from the end (<code>pop_back</code>)	0	$\Theta(1)$
insert at index i	$n - i$	$\Theta(n)$ worst case
remove at index i	$n - i - 1$	$\Theta(n)$ worst case
insert at the front (<code>push_front</code>)	n	$\Theta(n)$
remove from the front (<code>pop_front</code>)	$n - 1$	$\Theta(n)$

the cost is the number of elements that must move

16

Array insertions

0	1	2	3	4	5	6	7	8			n-2	n-1
23	-1	4	5	6	1	3			...			

size=7

push_back(45)

0	1	2	3	4	5	6	7	8			n-2	n-1
23	-1	4	5	6	1	3	45		...			

size=8

17

Array insertions

0	1	2	3	4	5	6	7	8			n-2	n-1
23	-1	4	5	6	1	3			...			

size=7

push_front(45)

0	1	2	3	4	5	6	7	8			n-2	n-1
23	23	-1	4	5	6	1	3		...			

size=7

0	1	2	3	4	5	6	7	8			n-2	n-1
45	23	-1	4	5	6	1	3		...			

size=8

18

Insert at

0	1	2	3	4	5	6	7	8			n-2	n-1
23	-1	4	5	6	1	3			...			

size=7

insert(4, 45)

0	1	2	3	4	5	6	7	8			n-2	n-1
23	-1	4	5	6	6	1	3		...			

size=7

0	1	2	3	4	5	6	7	8			n-2	n-1
23	-1	4	5	45	6	1	3		...			

size=8

```
// shift right, from the back
for (size_t j = size ; j > idx ; --j)
    A[j] = A[j - 1];
A[idx] = val;
++size;
```

19

Q5-5

• An array holds $n = 1,000,000$ elements. You call `pop_front` 1,000 times. Roughly how many element moves is that?

- A. About 10^3
- B. About 10^6
- C. About 10^9
- D. About 10^{12}
- E. None of the above

20

Dynamic (growing) arrays

Size vs capacity

- A growable array keeps **two** numbers

- ✓ **size**: elements stored
- ✓ **capacity**: slots allocated

0	1	2	3	4	5	6	7	8	9	10
23	-1	4	5	6	1	3				

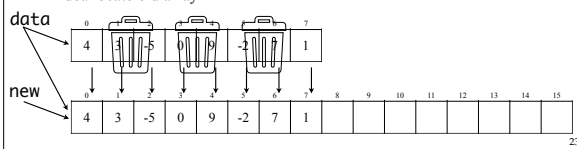
size=7 cap=11

- ✓ while **size** < **capacity**, **push_back** writes into a free slot
 - constant cost
- ✓ when **size** == **capacity**, there is no free slot
 - need to grow the array

22

Growing

- We cannot extend in place
 - ✓ memory right after the block may belong to someone else
 - ✓ every **grow** copies the whole array
- Triggered when the array is full (**size** == **capacity**)
 - allocate a new array with increased capacity
 - copy elements from old to new array
 - deallocate old array



23

Growing

```
void DynArray::grow(size_t new_cap) {
    int* bigger = new int[new_cap];
    for (size_t i = 0; i < m_size; ++i)
        bigger[i] = m_data[i]; // m_size copies
    delete[] m_data;
    m_data = bigger;
    m_capacity = new_cap;
}
```

24

Grow by one

- When array is full, grow to: **capacity + 1**

- Prove this strategy is $\Theta(n^2)$

- count element writes for n **appends**, starting empty
 - appending element k triggers $k - 1$ copies, then 1 write

append	copies	writes
1	0	1
2	1	1
3	2	1
4	3	1
5	4	1
...	...	...
n-1	n-2	1
n	n-1	1

$$\begin{aligned}T(n) &= n + \sum_{i=0}^{n-1} i \\&= n + \sum_{i=0}^{n-1} i \\&= n + \frac{n(n-1)}{2} \\&= \Theta(n^2) \quad \text{cost of appending } n \text{ elements}\end{aligned}$$

Appending n elements from empty costs $\Theta(n^2)$ in total. Per append, on average costs $\Theta(n)$.

25

Q5-6

- With *grow by one*, how many element **copies** does a 500-token reply cost?

- A. 500
- B. 1,000
- C. About 125,000
- D. About 250,000
- E. None of the above

26

Grow by a constant

- Idea: grow less often. When full, grow to: **capacity + c**
 - say $c = 1000$

- Consider $n = mc$ appends

- copies only happen at sizes $c, 2c, \dots, (m-1)c$
 - appending element k triggers $k - 1$ copies, then 1 write

$$\text{copies} = c(1 + 2 + \dots + (m-1)) = c \cdot \frac{m(m-1)}{2} = \frac{n^2}{2c} - \frac{n}{2} = \Theta(n^2)$$

c times fewer copies than grow by 1 but still quadratic cost

27

Q5-7

- With *grow by 1000*, what is the total number of **copies** for n consecutive appends?

- A. $\Theta(1)$
- B. $\Theta(n)$
- C. $\Theta(n \log n)$
- D. $\Theta(n^2)$
- E. None of the above

28